ME 314 Final Project - Jack Inside of Box (Default)

Spring 2025, Xing Yu Chen

In this project, I modeled and simulated the behavior of a planar jack-in-the-box system. The simulation captures the time-dependent dynamics of a multi-body system, specifically showing how the jack interacts with the box, including impacts between the jack and the box walls, as well as the effect of external forces.

Both rigid bodies (the jack and the box) are subject to gravity acting in the negative y-direction of the world frame. To maintain the position of the box, an upward force equal in magnitude to gravity is applied. Additionally, a sinusoidal torque is applied to the box to induce rotational motion, causing it to oscillate back and forth at a constant frequency.

Collaborators: Daniil Kniazkou

```
# All Imports
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt
from plotly.offline import init_notebook_mode, iplot
from IPython.display import display, HTML
import plotly.graph_objects as go
```

```
# Code Block for All Functions Used

# Create Transformation Function
def transform(angle, x, y):
    return sym.Matrix([
        [sym.cos(angle), -sym.sin(angle), 0, x],
        [sym.sin(angle), sym.cos(angle), 0, y],
        [0, 0, 1, 0],
        [0, 0, 0, 1]])

# Create Function to Unhat
def unhat(m):
  return sym.Matrix([m[0,3], m[1,3], m[2,3], m[2,1], m[0,2], m[1,0]])

# Create Function to Invert
def invert(m):
  R = sym.Matrix([
      [m[0,0], m[0,1], m[0,2]],
      [m[1,0], m[1,1], m[1,2]],
      [m[2,0], m[2,1], m[2,2]]]).T

  V_new = -R * sym.Matrix([m[0,3], m[1,3], m[2,3]])
  return sym.Matrix([
      [R[0,0], R[0,1], R[0,2], V_new[0]],
      [R[1,0], R[1,1], R[1,2], V_new[1]],
      [R[2,0], R[2,1], R[2,2], V_new[2]],
      [0, 0, 0, 1]])

# Create Translation Helper
def get_translation_component(g_matrix, component ="x"):

    # Map component to matrix row index
    row_map = {"x": 0, "y": 1, "z": 2}
    if component not in row_map:
        raise ValueError("Component must be one of 'x', 'y', or 'z'")
    row_idx = row_map[component]

    # Extract translation element from row `row_idx`, column 3, then substitute dummy symbols
    return g_matrix[row_idx, 3].subs(dum_dict)

def impact_update(s, impact_eqns, dum_list):
    """
    Update the system state vector `s` after an impact event.
    Solves the symbolic impact equations and substitutes the correct post-impact velocities.

    Parameters:
    - s: current state vector (positions and velocities)

    - impact_eqns: either a single sympy.Eq or a list of sympy.Eq equations representing impact constraints

    - dum_list: list of post-impact velocity symbols to solve for

    Returns:
    - Updated state vector as a numpy array with post-impact velocities substituted.
    """
```

```
    # Ensure "impact_eqns" is a List For Uniform Processing
    if isinstance(impact_eqns, sym.Eq):
        impact_eqns = [impact_eqns]

    # Build Substitution Dictionary From Current State (s)
    subs_dict = {
        x_b_l: s[0], y_b_l: s[1], theta_b_l: s[2],
        x_j_l: s[3], y_j_l: s[4], theta_j_l: s[5],
        x_b_ldot: s[6], y_b_ldot: s[7], theta_b_ldot: s[8],
        x_j_ldot: s[9], y_j_ldot: s[10], theta_j_ldot: s[11]}

    # Substitute Numerical Values Into Each Equation (lhs and rhs)
    numeric_eqns = []
    for eq in impact_eqns:
        lhs_num = eq.lhs.subs(subs_dict)
        rhs_num = eq.rhs.subs(subs_dict)
        numeric_eqns.append(sym.Eq(lhs_num, rhs_num))

    # Solve the Numeric Equations For Post-Impact Velocities + Lambda
    solutions = sym.solve(numeric_eqns, dum_list + [lamb], dict = True)

    if len(solutions) == 0:
        raise RuntimeError("No solutions found for impact equations.")

    elif len(solutions) == 1:
        # Only One Solution - Likely the Physically Valid One
        sol = solutions[0]

    else:
        # Multiple Solutions - Pick One With Non-Negligible Lambda
        sol = None
        for candidate in solutions:
            lambda_val = candidate.get(lamb, 0)
            if abs(lambda_val) > 1e-6:
                sol = candidate
                break
        if sol is None:
            sol = solutions[0] # Fallback to First Solution

    # Construct Updated State Vector: Positions Stay the Same; Velocities Updated
    s_updated = [s[0], s[1], s[2], s[3], s[4], s[5]]

    # Append Post-Impact Velocities in the Order of "dum_list"
    for symbol in dum_list:
        val = sol[symbol]
        s_updated.append(float(sym.N(val)))

    return np.array(s_updated)

def impact_condition(s, phi_func, threshold=1e-1):
    """
    Check for impact based on the values of impact constraint functions.

    Parameters:
        s (array-like): Current state vector with ordering
            [x_b, y_b, theta_b, x_j, y_j, theta_j,
             x_b_dot, y_b_dot, theta_b_dot, x_j_dot, y_j_dot, theta_j_dot]

        phi_func (function): Lambdified function evaluating impact constraints

        threshold (float): Small positive threshold for impact detection

    Returns:
        (bool, int or None): Tuple indicating if impact occurred and at which constraint index
    """
    # Evaluate the Constraints at the Current State
    phi_val = phi_func(*s)

    # "phi_val" Can be a List or Numpy Array - Convert to 1D Array For Iteration
    phi_arr = phi_val if isinstance(phi_val, (list, tuple)) else phi_val.flatten()

    # Check Each Constraint to See if it is Near Zero (Impact)
    for i, val in enumerate(phi_arr):
        if -threshold < val < threshold:
            return True, i # Impact detected at constraint i

    return False, None # No impact detected

# Create Integration Function
def integrate (f, xt, dt, time):
    k1 = dt * f(xt, time)
    k2 = dt * f(xt+k1/2., time)
    k3 = dt * f(xt+k2/2., time)
```

```
    k4 = dt * f(xt+k3, time)
    new_xt = xt + (1/6.) * (k1+2.0*k2+2.0*k3+k4)
    return new_xt

# Create Simulation Function
def simulate(f, s0, tspan, dt, integrate):
    N = int((max(tspan) - min(tspan)) / dt) + 1
    tvec = np.linspace(tspan[0], tspan[1], N)
    xtraj = np.zeros((len(s0), N))
    x = np.copy(s0)
    xtraj[:, 0] = s0

    for i in range(1, N):
        # Check for Impact
        impact, impact_num = impact_condition(x, phi_func, 1e-1)
        if impact:
            x = impact_update(x, impact_eqns_list[impact_num], dum_list)

        # Integrate Using Updated State
        xtraj[:, i] = integrate(f, x, dt, tvec[i - 1])
        x = np.copy(xtraj[:, i])

    return xtraj, tvec

# Dynamics Function (Returns State Derivative)
def dynamics(s, time):
    return np.array([
        *s[6:12],  # velocities (qdot)
        x_b_ddot_func(*s, time),
        y_b_ddot_func(*s, time),
        theta_b_ddot_func(*s, time),
        x_j_ddot_func(*s, time),
        y_j_ddot_func(*s, time),
        theta_j_ddot_func(*s, time)])
```

```
# Code Block For System Setup

# Define Constants
L_box, M_box = 6.0, 100.0
L_jack, M_jack = 1.0, 1.0
J_box = 4 * M_box * L_box**2
J_jack = 4 * M_jack * L_jack**2
g = 9.8
k = 30000

# Define Symbols
t = sym.symbols('t')

# Generalized Coordinates for the Box
x_b = sym.Function('x_b')(t)
y_b = sym.Function('y_b')(t)
theta_b = sym.Function('theta_b')(t)

# Generalized Coordinates for the Jack
x_j = sym.Function('x_j')(t)
y_j = sym.Function('y_j')(t)
theta_j = sym.Function('theta_j')(t)

# Configuration Vector (q) and its Derivatives
q = sym.Matrix([x_b, y_b, theta_b, x_j, y_j, theta_j])
qdot = q.diff(t)
qddot = qdot.diff(t)

# External Forces
theta_d = sym.sin(2 * sym.pi * t / 5)
F_y_b = 4 * M_box * g
F_theta_b = 0 # k * theta_d # abs(k * theta_d)

# Torque Vector
F_ext = sym.Matrix([0, F_y_b, F_theta_b, 0, 0, 0])

# Define All Transformations Symbolically
# Box Base (b) Pose in World (w) Frame
g_w_b = transform(theta_b, x_b, y_b)

# Box Wall Positions Relative to Box Center (b frame)
g_b_b1 = transform(0, L_box, L_box) # Box Wall 1 (front/top-right corner)
g_b_b2 = transform(0, 0, -L_box) # Box Wall 2 (bottom edge)
g_b_b3 = transform(0, -L_box, 0) # Box Wall 3 (left edge)
g_b_b4 = transform(0, 0, L_box) # Box Wall 4 (top edge)

# Jack Base (j) Pose in World Frame
```

```
g_w_j = transform(theta_j, x_j, y_j)

# Jack Corner Positions Relative to Jack Center (j frame)
g_j_j1 = transform(0, L_jack,  0) # Jack Corner 1 (right)
g_j_j2 = transform(0, 0, -L_jack) # Jack Corner 2 (bottom)
g_j_j3 = transform(0, -L_jack, 0) # Jack Corner 3 (left)
g_j_j4 = transform(0, 0, L_jack) # Jack Corner 4 (top)

# Frame-to-World Transformations for Box Walls
g_w_b1 = g_w_b * g_b_b1 # Wall 1 in World Frame
g_w_b2 = g_w_b * g_b_b2 # Wall 2 in World Frame
g_w_b3 = g_w_b * g_b_b3 # Wall 3 in World Frame
g_w_b4 = g_w_b * g_b_b4 # Wall 4 in World Frame

# Frame-to-World Transformations for Jack Corners
g_w_j1 = g_w_j * g_j_j1 # Corner 1 in World Frame
g_w_j2 = g_w_j * g_j_j2 # Corner 2 in World Frame
g_w_j3 = g_w_j * g_j_j3 # Corner 3 in World Frame
g_w_j4 = g_w_j * g_j_j4 # Corner 4 in World Frame

# Relative Transformations (Box Edge to Jack Corner)
# Wall 1 Relative to Jack Corners
g_b1_j1 = invert(g_w_b1) * g_w_j1
g_b1_j2 = invert(g_w_b1) * g_w_j2
g_b1_j3 = invert(g_w_b1) * g_w_j3
g_b1_j4 = invert(g_w_b1) * g_w_j4

# Wall 2 Relative to Jack Corners
g_b2_j1 = invert(g_w_b2) * g_w_j1
g_b2_j2 = invert(g_w_b2) * g_w_j2
g_b2_j3 = invert(g_w_b2) * g_w_j3
g_b2_j4 = invert(g_w_b2) * g_w_j4

# Wall 3 Relative to Jack Corners
g_b3_j1 = invert(g_w_b3) * g_w_j1
g_b3_j2 = invert(g_w_b3) * g_w_j2
g_b3_j3 = invert(g_w_b3) * g_w_j3
g_b3_j4 = invert(g_w_b3) * g_w_j4

# Wall 4 Relative to Jack Corners
g_b4_j1 = invert(g_w_b4) * g_w_j1
g_b4_j2 = invert(g_w_b4) * g_w_j2
g_b4_j3 = invert(g_w_b4) * g_w_j3
g_b4_j4 = invert(g_w_b4) * g_w_j4
```

```
# Code Block For Computational Work

# Body Velocities
V_box = unhat(invert(g_w_b) * g_w_b.diff(t))
V_jack = unhat(invert(g_w_j) * g_w_j.diff(t))

# Inertia Matrices
I_box = sym.diag(4*M_box, 4*M_box, 4*M_box, 0, 0, J_box)
I_jack = sym.diag(4*M_jack, 4*M_jack, 4*M_jack, 0, 0, J_jack)

# Potential Energy
PE = sym.simplify(g * (4 * M_box * y_b + 4 * M_jack * y_j))

# Kinetic Energy
KE = sym.simplify(0.5 * (V_box.T * I_box * V_box)[0] + 0.5 * (V_jack.T * I_jack * V_jack)[0])

# Lagrangian
L = sym.simplify(KE - PE)

# Euler-Lagrange Equations
dL_dq = sym.simplify(sym.Matrix([L]).jacobian(q).T)
dL_dqdot = sym.simplify(sym.Matrix([L]).jacobian(qdot).T)
ddL_dqdot_dt = sym.simplify(dL_dqdot.diff(t))
ELEQ = sym.simplify(sym.Eq(ddL_dqdot_dt - dL_dq, F_ext))

# Solve Euler-Lagrange Equations For "qddot"
sol = sym.solve(ELEQ, qddot, dict = True)

# Hamiltonian
H = sym.simplify((dL_dqdot.T * qdot)[0] - L)

# Lambdify Acceleration Functions For the Box
x_b_ddot_func = sym.lambdify([*q, *qdot, t], sol[0][qddot[0]])
y_b_ddot_func = sym.lambdify([*q, *qdot, t], sol[0][qddot[1]])
theta_b_ddot_func = sym.lambdify([*q, *qdot, t], sol[0][qddot[2]])

# Lambdify Acceleration Functions For the Jack
```

```
x_j_ddot_func = sym.lambdify([*q, *qdot, t], sol[0][qddot[3]])
y_j_ddot_func = sym.lambdify([*q, *qdot, t], sol[0][qddot[4]])
theta_j_ddot_func = sym.lambdify([*q, *qdot, t], sol[0][qddot[5]])

# Acceleration Matrix
qddot_Matrix = sym.Matrix([
    qdot[0], sol[0][qddot[0]],
    qdot[1], sol[0][qddot[1]],
    qdot[2], sol[0][qddot[2]],
    qdot[3], sol[0][qddot[3]],
    qdot[4], sol[0][qddot[4]],
    qdot[5], sol[0][qddot[5]]])
```

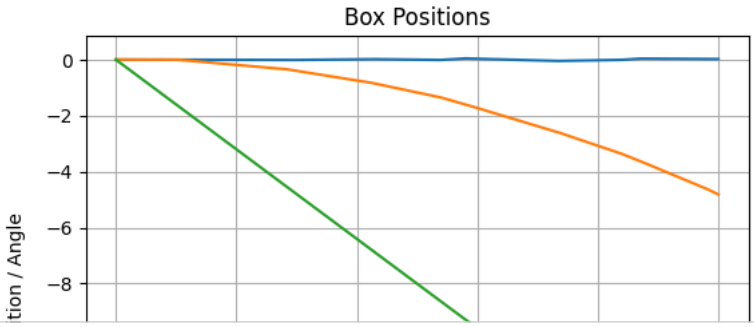
```
# Code Block For Dummy Symbol Substitutions

# Define Dummy Symbols For Lambdify
x_b_l, y_b_l, theta_b_l = sym.symbols('x_b_l y_b_l theta_b_l') # For the Box State
x_j_l, y_j_l, theta_j_l = sym.symbols('x_j_l y_j_l theta_j_l') # For the Jack State
x_b_ldot, y_b_ldot, theta_b_ldot = sym.symbols('x_b_ldot y_b_ldot theta_b_ldot') # For the Box Velocities
x_j_ldot, y_j_ldot, theta_j_ldot = sym.symbols('x_j_ldot y_j_ldot theta_j_ldot') # For the Jack Velocities

# Create Substitution Dictionary (Mapping Original 'q' and 'qdot' to Dummy Symbols)
dum_dict = {
    q[0]: x_b_l, q[1]: y_b_l, q[2]: theta_b_l,
    q[3]: x_j_l, q[4]: y_j_l, q[5]: theta_j_l,
    qdot[0]: x_b_ldot, qdot[1]: y_b_ldot, qdot[2]: theta_b_ldot,
    qdot[3]: x_j_ldot, qdot[4]: y_j_ldot, qdot[5]: theta_j_ldot}

# Substitute Dummy Symbols Into the Acceleration Matrix
qddot_dum = qddot_Matrix.subs(dum_dict)

# Lambdify the Matrix
qddot_lambdify = sym.lambdify(
    [x_b_l, x_b_ldot,
     y_b_l, y_b_ldot,
     theta_b_l, theta_b_ldot,
     x_j_l, x_j_ldot,
     y_j_l, y_j_ldot,
     theta_j_l, theta_j_ldot,
     t], qddot_dum)

# Substitute Dummy Symbols Into Hamiltonian
H_dum = H.subs(dum_dict)
dL_dqdot_dum = dL_dqdot.subs(dum_dict)

# Define Symbols For Post-Impact Generalized Velocities (τ+)
x_b_dot_plus, y_b_dot_plus, theta_b_dot_plus = sym.symbols('x_b_dot_plus y_b_dot_plus theta_b_dot_plus')
x_j_dot_plus, y_j_dot_plus, theta_j_dot_plus = sym.symbols('x_j_dot_plus y_j_dot_plus theta_j_dot_plus')
```

```
# Code Block For Impact Constraints

# Impact Constraints For Wall 1 (x-direction constraint)
phi_b1_j1 = get_translation_component(g_b1_j1, "x") # Right Wall to Jack Right
phi_b1_j2 = get_translation_component(g_b1_j2, "x") # Right Wall to Jack Bottom
phi_b1_j3 = get_translation_component(g_b1_j3, "x") # Right Wall to Jack Left
phi_b1_j4 = get_translation_component(g_b1_j4, "x") # Right Wall to Jack Top

# Impact Constraints For Wall 2 (y-direction constraint)
phi_b2_j1 = get_translation_component(g_b2_j1, "y") # Bottom Wall to Jack Right
phi_b2_j2 = get_translation_component(g_b2_j2, "y") # Bottom Wall to Jack Bottom
phi_b2_j3 = get_translation_component(g_b2_j3, "y") # Bottom Wall to Jack Left
phi_b2_j4 = get_translation_component(g_b2_j4, "y") # Bottom Wall to Jack Top

# Impact Constraints For Wall 3 (x-direction constraint)
phi_b3_j1 = get_translation_component(g_b3_j1, "x") # Left Wall to Jack Right
phi_b3_j2 = get_translation_component(g_b3_j2, "x") # Left Wall to Jack Bottom
phi_b3_j3 = get_translation_component(g_b3_j3, "x") # Left Wall to Jack Left
phi_b3_j4 = get_translation_component(g_b3_j4, "x") # Left Wall to Jack Top

# Impact Constraints For Wall 4 (y-direction constraint)
phi_b4_j1 = get_translation_component(g_b4_j1, "y") # Top Wall to Jack Right
phi_b4_j2 = get_translation_component(g_b4_j2, "y") # Top Wall to Jack Bottom
phi_b4_j3 = get_translation_component(g_b4_j3, "y") # Top Wall to Jack Left
phi_b4_j4 = get_translation_component(g_b4_j4, "y") # Top Wall to Jack Top

# Combine All Constraints Into a Single Matrix of Impact Constraints
phi_dum = sym.simplify(sym.Matrix([
    phi_b1_j1, phi_b1_j2, phi_b1_j3, phi_b1_j4, # Jack Relative to Box Wall 1
    phi_b2_j1, phi_b2_j2, phi_b2_j3, phi_b2_j4, # Jack Relative to Box Wall 2
    phi_b3_j1, phi_b3_j2, phi_b3_j3, phi_b3_j4, # Jack Relative to Box Wall 3
    phi_b4_j1, phi_b4_j2, phi_b4_j3, phi_b4_j4])) # Jack Relative to Box Wall 4
```

```
# Compute Jacobian of Impact Constraints
dphidq_dum = phi_dum.jacobian([
    x_b_l, y_b_l, theta_b_l,
    x_j_l, y_j_l, theta_j_l])

# Create Substitution Dictionary to Replace Pre-Impact Velocities With Post-Impact Velocity Symbols
impact_dict = {
    x_b_ldot: x_b_dot_plus, y_b_ldot: y_b_dot_plus, theta_b_ldot: theta_b_dot_plus,
    x_j_ldot: x_j_dot_plus, y_j_ldot: y_j_dot_plus, theta_j_ldot: theta_j_dot_plus}

# Apply Post-Impact Substitutions on dL_dqdot and phi Jacobian
dL_dqdot_dum_plus = sym.simplify(dL_dqdot_dum.subs(impact_dict))
dphidq_dum_plus = sym.simplify(dphidq_dum.subs(impact_dict))
H_dum_plus = sym.simplify(H_dum.subs(impact_dict))

# Define Lagrange Multiplier Symbol For Impact Constraints (λ)
lamb = sym.symbols('lambda')

# Initialize List to Hold Impact Equations
impact_eqns_list = []

# Define List of Post-Impact Velocity Symbols Consistent With Dummy Velocity Symbols
dum_list = [
    x_b_dot_plus, y_b_dot_plus, theta_b_dot_plus,
    x_j_dot_plus, y_j_dot_plus, theta_j_dot_plus]

# Construct the Left-Hand Side (Momentum and Energy Difference Terms)
lhs = sym.Matrix([
    dL_dqdot_dum_plus[0] - dL_dqdot_dum[0],
    dL_dqdot_dum_plus[1] - dL_dqdot_dum[1],
    dL_dqdot_dum_plus[2] - dL_dqdot_dum[2],
    dL_dqdot_dum_plus[3] - dL_dqdot_dum[3],
    dL_dqdot_dum_plus[4] - dL_dqdot_dum[4],
    dL_dqdot_dum_plus[5] - dL_dqdot_dum[5],
    H_dum_plus - H_dum])

# Loop Over Each Row of 'phi_dum' Constraints to Create Impact Equations
for i in range(phi_dum.shape[0]):
    rhs = sym.Matrix([
        lamb * dphidq_dum[i, 0],
        lamb * dphidq_dum[i, 1],
        lamb * dphidq_dum[i, 2],
        lamb * dphidq_dum[i, 3],
        lamb * dphidq_dum[i, 4],
        lamb * dphidq_dum[i, 5],
        0])
    # Equation: lhs == rhs
    impact_eqns_list.append(sym.simplify(sym.Eq(lhs, rhs)))

# Create a Lambdified Function For the Impact Constraints 'phi_dum'
phi_func = sym.lambdify(
    [
     x_b_l, y_b_l, theta_b_l,
     x_j_l, y_j_l, theta_j_l,
     x_b_ldot, y_b_ldot, theta_b_ldot,
     x_j_ldot, y_j_ldot, theta_j_ldot
    ], phi_dum)
```

```
# Code Block to Simulate the System and Plot Trajectories

# Setup For the Simulation
t_span = [0, 10] # Time Range
dt = 0.01 # Time Step
state_zero = np.array([0, 0, 0, 0, 0, 0, 0, 0, -1.6, 0, 0, 0]) # Initial State Vector

# Run Simulation
traj, tvec = simulate(dynamics, state_zero, t_span, dt, integrate)

# Plotting Box Positions
plt.figure()
plt.plot(tvec, traj[0, :], label ='x')
plt.plot(tvec, traj[1, :], label ='y')
plt.plot(tvec, traj[2, :], label ='theta')
plt.title('Box Positions')
plt.xlabel('Time [s]')
plt.ylabel('Position / Angle')
plt.legend()
plt.grid(True)
plt.show()

# Plotting Jack Positions
plt.figure()
```

```
plt.plot(tvec, traj[3, :], label ='x')
plt.plot(tvec, traj[4, :], label ='y')
plt.plot(tvec, traj[5, :], label ='theta')
plt.title('Jack Positions')
plt.xlabel('Time [s]')
plt.ylabel('Position / Angle')
plt.legend()
plt.grid(True)
plt.show()

# Plotting Box Velocities
plt.figure()
plt.plot(tvec, traj[6, :], label ='x_dot')
plt.plot(tvec, traj[7, :], label ='y_dot')
plt.plot(tvec, traj[8, :], label ='theta_dot')
plt.title('Box Velocities')
plt.xlabel('Time [s]')
plt.ylabel('Velocity / Angular Velocity')
plt.legend()
plt.grid(True)
plt.show()

# Plotting Jack Velocities
plt.figure()
plt.plot(tvec, traj[9, :], label ='x_dot')
plt.plot(tvec, traj[10, :], label ='y_dot')
plt.plot(tvec, traj[11, :], label ='theta_dot')
plt.title('Jack Velocities')
plt.xlabel('Time [s]')
plt.ylabel('Velocity / Angular Velocity')
plt.legend()
plt.grid(True)
plt.show()
```




```
# Code Block to Animate the System

def animate_jack_in_a_box(config_array, l_box = 1, l_jack = 0.2, T = 10):
    # Setup Plotly Display Settings For Jupyter Notebook
    def configure_plotly_browser_state():
        display(HTML('''
            <script src ="/static/components/requirejs/require.js"></script>
            <script>
              requirejs.config({
                paths: {
                  base: '/static/base',
                  plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
                },
              });
            </script>
        '''))
    configure_plotly_browser_state()
    init_notebook_mode(connected = False)

    # Unpack State Trajectories
    N = config_array.shape[1]
    x_b, y_b, th_b = config_array[0], config_array[1], config_array[2] # Box Position and Angle
    x_j, y_j, th_j = config_array[3], config_array[4], config_array[5] # Jack Position and Angle

    # Function to Build a Homogeneous Transformation Matrix in 2D (With Numpy)
    def transform_np(x, y, theta):
        return np.array([
            [np.cos(theta), -np.sin(theta), 0, x],
            [np.sin(theta), np.cos(theta), 0, y],
            [0, 0, 1, 0],
            [0, 0, 0, 1]])

    # Preallocate Arrays For Transformed Coordinates
    b_walls = np.zeros((4, 2, N)) # Box Has 4 Walls
    j_corners = np.zeros((5, 2, N)) # Jack Has 1 Center + 4 Corners

    # Loop Through Each Time Step to Compute World-Frame Positions
    for i in range(N):
        g_b = transform_np(x_b[i], y_b[i], th_b[i]) # Transform for the Box
        g_j = transform_np(x_j[i], y_j[i], th_j[i]) # Transform for the Jack

        # Define Box Walls in Local Frame
        box = np.array([
            [ l_box,  l_box, 0, 1],
            [ l_box, -l_box, 0, 1],
            [-l_box, -l_box, 0, 1],
            [-l_box,  l_box, 0, 1],]).T

        # Define Jack Corners in Local Frame
        jack = np.array([
            [ 0,  l_jack,   0,  -l_jack,  0],   # x-coordinates
            [ 0,    0,  -l_jack,   0,  l_jack], # y-coordinates
            [ 0,    0,      0,     0,     0],   # z = 0
            [ 1,    1,      1,     1,     1]])  # Homogeneous Coordinates

        # Transform Box and Jack Into World Frame
        box_world = g_b @ box
        jack_world = g_j @ jack

        # Store the 2D Transformed Coordinates For Plotting
        for j in range(4):
            b_walls[j, :, i] = box_world[:2, j]

        for j in range(5):
            j_corners[j, :, i] = jack_world[:2, j]

    # Set Axis Limits
    xm, xM = np.min(b_walls[:, 0]) - 2, np.max(b_walls[:, 0]) + 2
    ym, yM = np.min(b_walls[:, 1]) - 2, np.max(b_walls[:, 1]) + 2
```

```
    # Initialize the Static Plot (First Frame)
    data = [
        # Draw the Box
        go.Scatter(x = b_walls[:, 0, 0].tolist() + [b_walls[0, 0, 0]],
                   y = b_walls[:, 1, 0].tolist() + [b_walls[0, 1, 0]],
                   mode ='lines', line = dict(color ='black', width = 3),
                   name ='Box'),

        # Draw the Jack
        go.Scatter(x = [j_corners[i, 0, 0] for i in [1, 3, 0, 2, 4]],
                   y = [j_corners[i, 1, 0] for i in [1, 3, 0, 2, 4]],
                   mode ='lines', line = dict(color ='red', width = 3),
                   name ='Jack'),

        # Draw Markers at Jack Tips
        go.Scatter(x = [j_corners[i, 0, 0] for i in [1, 2, 3, 4]],
                   y = [j_corners[i, 1, 0] for i in [1, 2, 3, 4]],
                   mode ='markers', marker = dict(color ='darkred', size = 6),
                   name ='Jack Ends')]

    # Configure Plot Layout and Animation Controls
    layout = dict(
        xaxis = dict(range = [xm, xM], autorange = False, zeroline = False, dtick = 1),
        yaxis = dict(range = [ym, yM], autorange = False, zeroline = False, scaleanchor ='x', dtick = 1),
        title ='Jack in a Box Simulation',
        hovermode ='closest',
        updatemenus = [dict(type ='buttons',
                        buttons = [dict(label ='Play', # Add a Play Button
                                    method ='animate',
                                    args = [None, dict(frame = dict(duration = 1000*T//N, redraw = False),
                                                    fromcurrent = True, mode ='immediate')]),

                                dict(label ='Pause', # Add a Pause Button
                                    method ='animate',
                                    args = [[None],
                                          dict(frame = dict(duration = 0, redraw = False),
                                               mode ='immediate',
                                               transition = dict(duration = 0))])])])

    # Create Animation Frames
    frames = []
    for k in range(N):
        frame = go.Frame(data = [
            # Update Box
            go.Scatter(x = b_walls[:, 0, k].tolist() + [b_walls[0, 0, k]],
                       y = b_walls[:, 1, k].tolist() + [b_walls[0, 1, k]],
                       mode ='lines', line = dict(color ='black', width = 3)),

            # Update Jack
            go.Scatter(x = [j_corners[i, 0, k] for i in [1, 3, 0, 2, 4]],
                       y = [j_corners[i, 1, k] for i in [1, 3, 0, 2, 4]],
                       mode ='lines', line = dict(color ='red', width = 3)),

            # Update Jack Tips
            go.Scatter(x = [j_corners[i, 0, k] for i in [1, 2, 3, 4]],
                       y = [j_corners[i, 1, k] for i in [1, 2, 3, 4]],
                       mode ='markers', marker = dict(color ='darkred', size = 6))])
        frames.append(frame)

    # Assemble and Display Figure With Animation
    fig = go.Figure(data = data, layout = layout, frames = frames)
    iplot(fig)

# Call the Animation Function
```